LAPORAN PRAKTIKUM
TUGAS BAB 6 Skenario logging dari aplikasi PMB

Dosen Pengampu: Ferry Astika Saputra, S.T., M.Sc.



Disusun Oleh:

Adam Rasyid Nurmuhammad

3126640025

D4-IT-B

PROGRAM STUDI SARJANA TERAPAN

JURUSAN TEKNIK INFORMATIKA

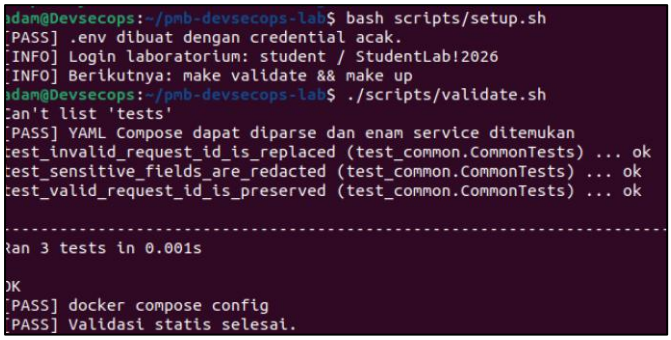
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER

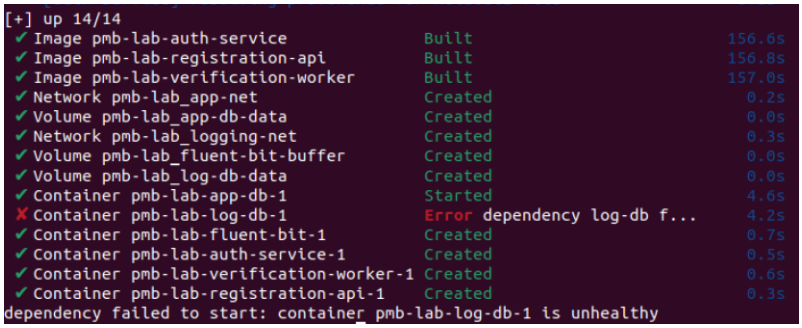
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2026

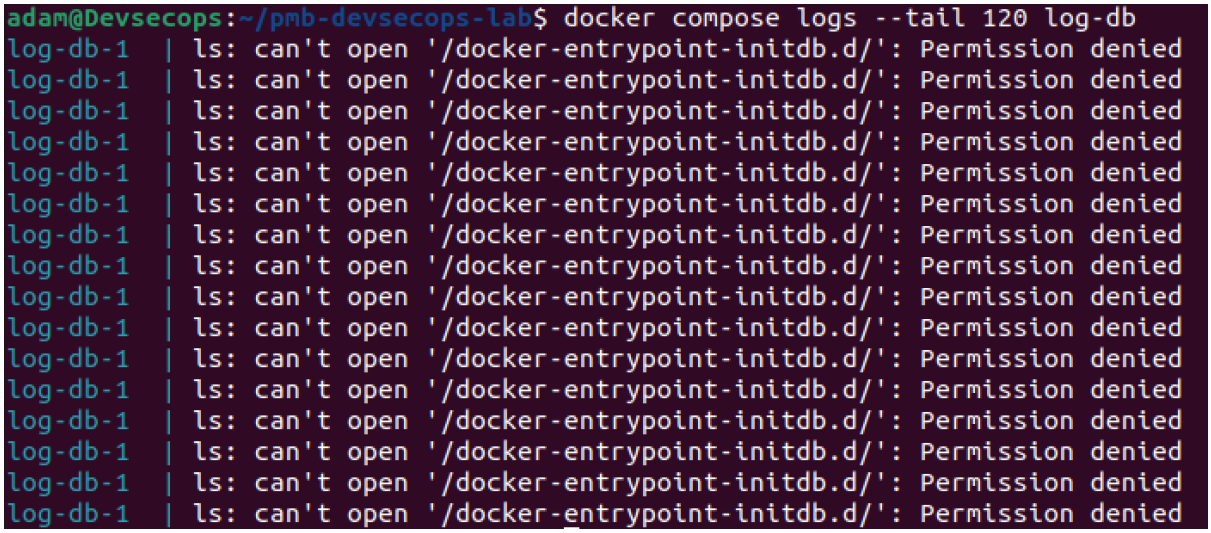
0. PERSIAPAN

bash scripts/setup.sh, ./scripts/validate.sh, docker compose up -d –build





Terjadi service unhealthy di pmb-lab-log-db-1, Saat di cek dengan logs



container log-db tidak punya izin membaca folder init-log yang dipasang ke /docker-entrypoint-initdb.d. Image PostgreSQL memeriksa dan menjalankan skrip di folder itu saat inisialisasi.

Solusinya:

sudo chmod 755 init-log init-app

sudo chmod 755 init-log/*.sh init-app/*.sh

docker compose up -d

docker compose ps

```
adam@Devsecops:~/pmb-devsecops-lab$ sudo chmod 755 init-log init-app
[sudo] password for adam:
adam@Devsecops:~/pmb-devsecops-lab$ sudo chmod 755 init-log/*.sh init-app/*.sh
adam@Devsecops:~/pmb-devsecops-lab$ docker compose up -d
[+] up 2/2
 ✔ Container pmb-lab-app-db-1 Running
 ✘ Container pmb-lab-log-db-1 Error dependency log-db failed to start
dependency failed to start: container pmb-lab-log-db-1 is unhealthy
adam@Devsecops:~/pmb-devsecops-lab$ docker compose ps
NAME                IMAGE                COMMAND                  SERVICE   CREATED          STATUS                             PORTS
pmb-lab-app-db-1    postgres:16-alpine   "docker-entrypoint.s…"   app-db    12 minutes ago   Up 19 seconds (healthy)            5432/tcp
pmb-lab-log-db-1    postgres:16-alpine   "docker-entrypoint.s…"   log-db    12 minutes ago   Restarting (1) 31 seconds ago
```

app-db sudah **healthy**, tetapi log-db masih **Restarting**, sehingga service lainnya belum dapat mulai. Coba jalankan docker compose logs --tail 80 log-db untuk melihat message **terbaru**,

```
log-db-1  | PostgreSQL init process complete; ready for start up.
log-db-1  |
log-db-1  | 2026-10-27 12:44:33.931 UTC [1] LOG:  starting PostgreSQL 16.15 on x86_64-pc-linux-musl, compiled
log-db-1  | 2026-10-27 12:44:33.932 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
log-db-1  | 2026-10-27 12:44:33.932 UTC [1] LOG:  listening on IPv6 address "::", port 5432
log-db-1  | 2026-10-27 12:44:33.947 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
log-db-1  | 2026-10-27 12:44:33.968 UTC [65] LOG:  database system was shut down at 2026-10-27 12:44:33 UTC
log-db-1  | 2026-10-27 12:44:33.987 UTC [1] LOG:  database system is ready to accept connections
```

Berhasil memuat service db

Namun ternyata ada error saat docker compose up -d

```
adam@Devsecops:~/pmb-devsecops-lab$ docker compose up -d
[+] up 5/5
 ✔ Container pmb-lab-app-db-1              Healthy
 ✔ Container pmb-lab-log-db-1              Healthy
 ✔ Container pmb-lab-fluent-bit-1          Started
 ✘ Container pmb-lab-auth-service-1        Error dependency auth-service failed to start
 ✔ Container pmb-lab-verification-worker-1 Started
dependency failed to start: container pmb-lab-auth-service-1 is unhealthy
```

Jalankan docker compose logs --tail 100 auth-service

```
auth-service-1  | PermissionError: [Errno 13] Permission denied: '/app/auth_service.py'
auth-service-1  | [Errno 13] Permission denied: '/app/auth_service.py'
auth-service-1  | [2026-10-27 12:55:30 +0000] [8] [INFO] Worker exiting (pid: 8)
auth-service-1  | [2026-10-27 12:55:30 +0000] [1] [ERROR] Control server error: [Errno 30]
auth-service-1  | [2026-10-27 12:55:30 +0000] [1] [ERROR] Worker (pid:7) exited with code 3
auth-service-1  | [2026-10-27 12:55:30 +0000] [1] [ERROR] Worker (pid:8) exited with code 3
```

Ternyata blm ada permission, tambahkan permissionnya

```
adam@Devsecops:~/pmb-devsecops-lab$ chmod -R a+rX app
```

Coba build Kembali docker compose up -d –build

```
[+] up 9/9
 ✔ Image pmb-lab-auth-service              Built
 ✔ Image pmb-lab-registration-api          Built
 ✔ Image pmb-lab-verification-worker       Built
 ✔ Container pmb-lab-fluent-bit-1          Running
 ✔ Container pmb-lab-app-db-1              Healthy
 ✔ Container pmb-lab-log-db-1              Healthy
 ✔ Container pmb-lab-auth-service-1        Healthy
 ✔ Container pmb-lab-verification-worker-1 Started
 ✔ Container pmb-lab-registration-api-1    Started
```

Ternyata masih error saat curl

```
adam@Devsecops:~/pmb-devsecops-lab$ curl -fsS http://127.0.0.1:8080/healthz
curl: (7) Failed to connect to 127.0.0.1 port 8080 after 0 ms: Connection refused
```

aplikasinya sudah hidup dan sehat di dalam container, tetapi port 8080 tidak tersambung ke VM. Karena registration-api hanya berada di jaringan Docker internal, Docker mencatat pengaturan portnya tetapi tidak benar-benar membukanya. Itulah sebabnya pemeriksaan kesehatan di dalam container berhasil, sementara curl 127.0.0.1:8080 dari terminal VM mendapat *Connection refused*.

buka compose.yaml dan tambahkan satu jaringan untuk dua layanan yang perlu diakses dari VM:

```
 registration-api:
   build:
     context: ./app
   environment:
     <<: *app-environment
     AUTH_URL: ${AUTH_URL}
     SESSION_SECRET: ${SESSION_SECRET}
   ports:
     - "127.0.0.1:8080:8080"
   networks:
     - app-net
     - host-access
```

```
fluent-bit:
  image: fluent/fluent-bit:4.2.8
  environment:
    LOG_DB_NAME: ${LOG_DB_NAME}
    FLUENTBIT_DB_USER: ${FLUENTBIT_DB_USER}
    FLUENTBIT_DB_PASSWORD: ${FLUENTBIT_DB_PASSWORD}
  volumes:
    - ./fluent-bit/fluent-bit.conf:/fluent-bit/etc/fluent-bit.conf:ro
    - fluent-bit-buffer:/var/log/flb-storage
  ports:
    - "127.0.0.1:24224:24224"
    - "127.0.0.1:2020:2020"
  networks:
    - logging-net
    - host-access
```

```
networks:
  app-net:
    internal: true
  logging-net:
    internal: true
  hostaccess: {}
```

```
adam@Devsecops:~/pmb-devsecops-lab$ docker compose ps registration-api fluent-bit
curl -fsS http://127.0.0.1:8080/healthz
NAME                          IMAGE                       COMMAND                  SER
TS
pmb-lab-fluent-bit-1          fluent/fluent-bit:4.2.8     "/fluent-bit/bin/flu…"   flu
0/tcp, 24224/tcp
pmb-lab-registration-api-1    pmb-lab-registration-api    "gunicorn --bind 0.0…"   reg
.0.0.1:8080->8080/tcp, 8081/tcp
{"service":"registration-api","status":"ok"}
```

Langkah selanjutnya Adalah cek url dari /health untuk fluent-bitnya

```
adam@Devsecops:~/pmb-devsecops-lab$ curl -fsS http://127.0.0.1:2020/api/v1/health
{"fluent-bit":{"version":"4.2.8","edition":"Community","flags":["FLB_HAVE_SYS_WAIT_H","FLB_HAVE_SIMD","FLB_HAVE_IN_STORAGE_BACKLOG","FLB_HAVE_CHUNK_TR
ACE","FLB_HAVE_PARSER","FLB_HAVE_RECORD_ACCESSOR","FLB_HAVE_STREAM_PROCESSOR","FLB_HAVE_UNICODE_ENCODER","FLB_EVENT_LOOP_AUTO_DISCOVERY","FLB_HAVE_TLS
","FLB_HAVE_OPENSSL","FLB_HAVE_METRICS","FLB_HAVE_PROFILES","FLB_HAVE_WASM","FLB_HAVE_KAFKA_SASL","FLB_HAVE_KAFKA_OAUTHBEARER","FLB_HAVE_AWS_MSK_IAM",
"FLB_HAVE_AWS","FLB_HAVE_AWS_CREDENTIAL_PROCESS","FLB_HAVE_SIGNV4","FLB_HAVE_SQLDB","FLB_LOG_NO_CONTROL_CHARS","FLB_HAVE_METRICS","FLB_HAVE_HTTP_SERVE
R","FLB_HAVE_SYSTEMD","FLB_HAVE_SYSTEMD_SDBUS","FLB_HAVE_FORK","FLB_HAVE_GMTOFF","FLB_HAVE_TIME_ZONE","FLB_HAVE_UNIX_SOCKET","FLB_HAVE_LITTLE_ENDIAN_S
YSTEM","FLB_HAVE_LIBYAML","FLB_HAVE_ATTRIBUTE_ALLOC_SIZE","FLB_HAVE_PROXY_GO","FLB_HAVE_JEMALLOC","FLB_HAVE_LIBBACKTRACE","FLB_HAVE_REGEX","FLB_HAVE_U
TF8_ENCODER","FLB_HAVE_LUAJIT","FLB_HAVE_ACCEPT4","FLB_HAVE_INOTIFY","FLB_HAVE_GETENTROPY","FLB_HAVE_GETENTROPY_SYS_RANDOM"]}}adam@Devsecops:~/pmb-dev
secops-lab$ docker compose ps fluent-bit
NAME                   IMAGE                       COMMAND                  SERVICE      CREATED              STATUS              PORTS
pmb-lab-fluent-bit-1   fluent/fluent-bit:4.2.8     "/fluent-bit/bin/flu…"   fluent-bit   About a minute ago   Up About a minute   127.0.0.1:2020->2020/t
cp, 127.0.0.1:24224->24224/tcp
```

/api/v1/health berhasil dimuat, kemudian jalankan verify.sh untuk cek seluruh stacknya

```
adam@Devsecops:~/pmb-devsecops-lab$ ./scripts/verify.sh
NAME                              IMAGE                              COMMAND
     PORTS
pmb-lab-app-db-1                  postgres:16-alpine                 "docker-entrypoint
y)   5432/tcp
pmb-lab-auth-service-1            pmb-lab-auth-service               "gunicorn --bind 0
y)   8080-8081/tcp
pmb-lab-fluent-bit-1              fluent/fluent-bit:4.2.8            "/fluent-bit/bin/f
     127.0.0.1:2020->2020/tcp, 127.0.0.1:24224->24224/tcp
pmb-lab-log-db-1                  postgres:16-alpine                 "docker-entrypoint
y)   5432/tcp
pmb-lab-registration-api-1        pmb-lab-registration-api           "gunicorn --bind 0
)    127.0.0.1:8080->8080/tcp, 8081/tcp
pmb-lab-verification-worker-1     pmb-lab-verification-worker        "python worker.py"
     8080-8081/tcp
[PASS] Fluent Bit siap
[PASS] Portal PMB siap
[INFO] Pendaftaran: 0
[INFO] Record log: 99
[FAIL] Belum ada transaksi. Jalankan make normal dan make incident.
```

Semua layanan siap, portal dan Fluent Bit merespons, serta 99 record log sudah tersimpan.

1. TAHAP A - Normal (Baseline)

Jalankan

./scripts/set-incident.sh none

./scripts/demo-normal.sh

```
adam@Devsecops:~/pmb-devsecops-lab$ ./scripts/set-incident.sh none
./scripts/demo-normal.sh
UPDATE 1
      key      | value |          updated_at
---------------+-------+-------------------------------
 incident_mode | none  | 2026-10-27 13:37:27.554037+00
(1 row)

[PASS] Mode insiden: none
[PASS] Portal PMB siap
[INFO] Request ID: REQ-b327d759254442a39b01a73a602b4033
             public_id               |  status  | attempt | failure_code |              request_id
-------------------------------------+----------+---------+--------------+--------------------------------------
 20c35dcd-e1ca-48f3-b209-5e1ef2a16e8d | VERIFIED |       1 |              | REQ-b327d759254442a39b01a73a602b4033
(1 row)

[PASS] Demo normal selesai. Status yang diharapkan: VERIFIED.
```

Pastikan hasil transaksi berstatus VERIFIED, lalu catat *request ID*-nya. Setelah itu jalankan lagi verify.sh

```
adam@Devsecops:~/pmb-devsecops-lab$ ./scripts/verify.sh
NAME                          IMAGE                          COMMAND                  SERVICE              CREATED             STATUS
     PORTS
pmb-lab-app-db-1              postgres:16-alpine             "docker-entrypoint.s…"   app-db               About an hour ago   Up 55 minutes (health
y)   5432/tcp
pmb-lab-auth-service-1        pmb-lab-auth-service           "gunicorn --bind 0.0…"   auth-service         36 minutes ago      Up 36 minutes (health
y)   8080-8081/tcp
pmb-lab-fluent-bit-1          fluent/fluent-bit:4.2.8        "/fluent-bit/bin/flu…"   fluent-bit           9 minutes ago       Up 9 minutes
     127.0.0.1:2020->2020/tcp, 127.0.0.1:24224->24224/tcp
pmb-lab-log-db-1              postgres:16-alpine             "docker-entrypoint.s…"   log-db               About an hour ago   Up 54 minutes (health
y)   5432/tcp
pmb-lab-registration-api-1    pmb-lab-registration-api       "gunicorn --bind 0.0…"   registration-api     14 minutes ago      Up 14 minutes (health
y)   127.0.0.1:8080->8080/tcp, 8081/tcp
pmb-lab-verification-worker-1 pmb-lab-verification-worker    "python worker.py"       verification-worker  36 minutes ago      Up 36 minutes
     8080-8081/tcp
[PASS] Fluent Bit siap
[PASS] Portal PMB siap
[INFO] Pendaftaran: 1
[INFO] Record log: 300
[PASS] Service sehat, transaksi tersedia, log tersimpan, dan pemeriksaan secret bersih.
```

Simpan Output tersebut sebagai bukti baseline

Tahap A berhasil: transaksi berstatus VERIFIED, *request ID* REQ-b327d759254442a39b01a73a602b4033

2. TAHAP B - Failure injection:

```
adam@Devsecops:~/pmb-devsecops-lab$ ./scripts/demo-incident.sh
[PASS] Portal PMB siap
[INFO] Request ID: REQ-90d7301bad1c4cc9a26867730132913c
             public_id               | status | attempt |  failure_code  |              request_id
-------------------------------------+--------+---------+----------------+-------------------------------------
a8a7d1a6-7525-46c6-8c1a-dfd13b0fd9cb | FAILED |       3 | VERIFY-TIMEOUT | REQ-90d7301bad1c4cc9a26867730132913c
(1 row)

[INFO] Gunakan request ID di atas pada queries/incident-analysis.sql.
[PASS] Demo insiden selesai. Status yang diharapkan: FAILED / VERIFY-TIMEOUT.
```

```
        time          |       service       |        event_name        | severity | attempt |   error_code
----------------------+---------------------+--------------------------+----------+---------+----------------
2026-10-27 13:45:13 | registration-api    | registration_submitted | INFO     |         |
2026-10-27 13:45:13 | registration-api    | http_request           | INFO     |         |
2026-10-27 13:45:13 | verification-worker | verification_started   | INFO     |         |
2026-10-27 13:45:13 | verification-worker | verification_retry     | WARN     | 1       |
2026-10-27 13:45:14 | verification-worker | verification_retry     | WARN     | 2       |
2026-10-27 13:45:15 | verification-worker | verification_retry     | WARN     | 3       |
2026-10-27 13:45:16 | verification-worker | dependency_timeout     | ERROR    | 3       | VERIFY-TIMEOUT
```

```
2026-10-27 13:45:28 | registration-api    | INFO     | http_request           | REQ-8f8f007df013452789de1afcdff9fcf5 |
      |
2026-10-27 13:45:22 | registration-api    | INFO     | http_request           | REQ-29899936fade43d480d3e49dbf70ad0b |
      |
2026-10-27 13:45:16 | registration-api    | INFO     | http_request           | REQ-407395ca65f5468b966f90191cb817be |
      |
2026-10-27 13:45:16 | verification-worker | ERROR    | dependency_timeout     | REQ-90d7301bad1c4cc9a26867730132913c | a8a7d1a6-7525-46c6-8c1a-dfd13b
fd9cb | VERIFY-TIMEOUT
2026-10-27 13:45:15 | verification-worker | WARN     | verification_retry     | REQ-90d7301bad1c4cc9a26867730132913c | a8a7d1a6-7525-46c6-8c1a-dfd13b
fd9cb |
2026-10-27 13:45:14 | verification-worker | WARN     | verification_retry     | REQ-90d7301bad1c4cc9a26867730132913c | a8a7d1a6-7525-46c6-8c1a-dfd13b
fd9cb |
2026-10-27 13:45:13 | registration-api    | INFO     | registration_submitted | REQ-90d7301bad1c4cc9a26867730132913c | a8a7d1a6-7525-46c6-8c1a-dfd13b
fd9cb |
2026-10-27 13:45:13 | registration-api    | INFO     | http_request           | REQ-90d7301bad1c4cc9a26867730132913c |
      |
2026-10-27 13:45:13 | verification-worker | INFO     | verification_started   | REQ-90d7301bad1c4cc9a26867730132913c | a8a7d1a6-7525-46c6-8c1a-dfd13b
fd9cb |
2026-10-27 13:45:13 | verification-worker | WARN     | verification_retry     | REQ-90d7301bad1c4cc9a26867730132913c | a8a7d1a6-7525-46c6-8c1a-dfd13b
```

Pada transaksi dengan *request ID* REQ-90d7301bad1c4cc9a26867730132913c, pendaftaran dikirim pada 13:45:13 dan mulai diproses oleh verification-worker. Service tersebut menjadi sumber peringatan pertama melalui event verification_retry pada percobaan 1, kemudian mengulangi percobaan hingga 3 kali. Pada 13:45:16 muncul event dependency_timeout dengan tingkat ERROR dan kode VERIFY-TIMEOUT. Akibatnya, status transaksi menjadi FAILED dengan attempt = 3. Pemeriksaan log tidak menemukan indikasi kebocoran *secret* (0 temuan).

3. Tahap C — Recovery

Jalankan

./scripts/set-incident.sh none

./scripts/demo-normal.sh

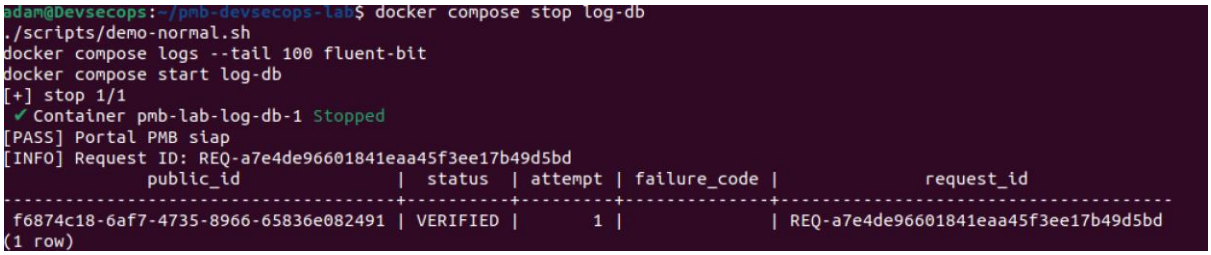
```
adam@Devsecops:~/pmb-devsecops-lab$ ./scripts/set-incident.sh none
./scripts/demo-normal.sh
UPDATE 1
      key      | value |          updated_at
---------------+-------+-------------------------------
 incident_mode | none  | 2026-10-27 13:53:50.292844+00
(1 row)

[PASS] Mode insiden: none
[PASS] Portal PMB siap
[INFO] Request ID: REQ-f7b76e87e6004a5ca48dcc206d85873a
             public_id               |  status  | attempt | failure_code |              request_id
-------------------------------------+----------+---------+--------------+-------------------------------------
 1c106648-1f1c-416f-acf1-f89df630430a | VERIFIED |       1 |              | REQ-f7b76e87e6004a5ca48dcc206d85873a
(1 row)

[PASS] Demo normal selesai. Status yang diharapkan: VERIFIED.
adam@Devsecops:~/pmb-devsecops-lab$ set -a; source .env; set +a
docker compose exec -T app-db psql -U "$APP_DB_ADMIN_USER" -d "$APP_DB_NAME" -c "SELECT request_id, status, attempt, failure_code FROM registrations W
HERE request_id IN ('REQ-90d7301bad1c4cc9a26867730132913c', 'REQ-f7b76e87e6004a5ca48dcc206d85873a');"
             request_id             |  status  | attempt |  failure_code
------------------------------------+----------+---------+----------------
 REQ-90d7301bad1c4cc9a26867730132913c | FAILED   |       3 | VERIFY-TIMEOUT
 REQ-f7b76e87e6004a5ca48dcc206d85873a | VERIFIED |       1 |
(2 rows)
```

Setelah mode insiden dikembalikan ke none, transaksi baru dengan *request ID* REQ-f7b76e87e6004a5ca48dcc206d85873a berhasil mencapai status VERIFIED dalam satu percobaan. Hasil ini menunjukkan layanan kembali memproses pendaftaran secara normal. Transaksi pada Tahap B yang sudah FAILED tidak otomatis diproses ulang; pemulihan dibuktikan melalui transaksi baru.

4. Tahap D — Resilience logging



Saat log-db dihentikan, transaksi REQ-a7e4de96601841eaa45f3ee17b49d5bd tetap mencapai status VERIFIED dalam satu percobaan. Fluent Bit gagal mengirim log ke PostgreSQL dan menampilkan pesan failed to flush chunk disertai jadwal retry. Setelah log-db dijalankan kembali, keberadaan log transaksi tersebut perlu diperiksa di database log. Pengujian ini tidak membuktikan *exactly-once delivery*.

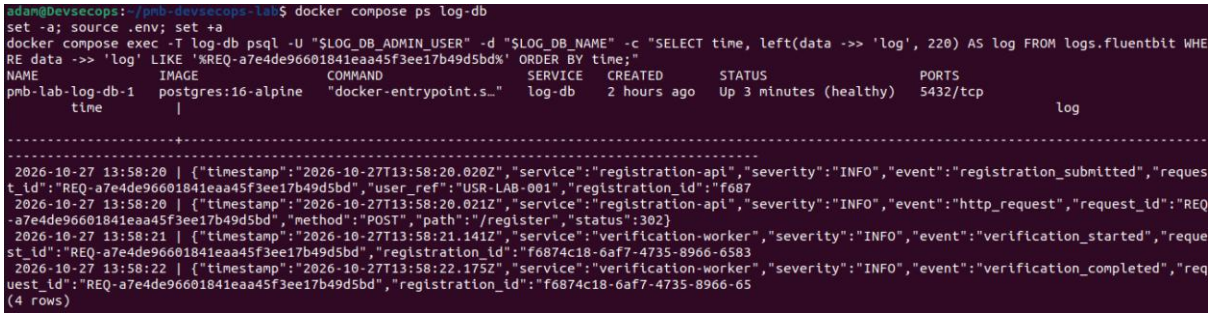
- Amati retry dan penggunaan buffer Fluent Bit.

Jalankan

docker compose ps log-db

set -a; source .env; set +a

docker compose exec -T log-db psql -U "$LOG_DB_ADMIN_USER" -d "$LOG_DB_NAME" -c "SELECT time, left(data ->> 'log', 220) AS log FROM logs.fluentbit WHERE data ->> 'log' LIKE '%REQ-a7e4de96601841eaa45f3ee17b49d5bd%' ORDER BY time;"



Saat log-db dihentikan, transaksi REQ-a7e4de96601841eaa45f3ee17b49d5bd tetap berhasil dengan status VERIFIED. Fluent Bit menampilkan kegagalan pengiriman log dan melakukan retry. Setelah log-db kembali sehat, empat event transaksi tersebut ditemukan di logs.fluentbit, sehingga backlog terbukti terkirim. Pengujian ini belum membuktikan *exactly-once delivery*.

Pertanyaan Analitis

1. **Mengapa status container Up tidak membuktikan fungsi bisnis sehat?**
   Up hanya menunjukkan proses container berjalan. Pada Tahap B, layanan tetap berjalan, tetapi verifikasi pendaftaran gagal setelah tiga percobaan dan transaksi berakhir FAILED / VERIFY-TIMEOUT. Fungsi bisnis perlu diperiksa melalui status transaksi dan kronologi event.
2. **Apa perbedaan tiga waktu pada log?**
   Event time adalah saat aplikasi membuat event, yang tercantum pada timestamp di structured log. Waktu penerimaan collector adalah saat Fluent Bit menerima event, sedangkan waktu penyimpanan database adalah saat PostgreSQL berhasil menyimpannya. Ketiganya dapat berbeda saat terjadi antrean dan retry; skema lab belum menyimpan ketiganya sebagai kolom terpisah.
3. **Mengapa nama dan email tidak dimasukkan ke structured log?**
   Nama dan email adalah data pribadi yang tidak diperlukan untuk menelusuri alur insiden. Proyek memakai request_id, registration_id, nama event, dan status agar transaksi tetap dapat ditelusuri tanpa menyalin data pribadi ke database log.
4. **Apa risiko request_id dari client tanpa validasi?**
   Client dapat mengirim nilai yang terlalu panjang atau berisi karakter tak terduga, sehingga pencarian dan korelasi log menjadi tidak andal. Proyek memvalidasi formatnya dan membuat ID baru bila nilai yang diterima tidak valid.
5. **Apa yang terjadi bila buffer Fluent Bit mencapai 256 MB?**
   Batas storage.total_limit_size 256M membatasi backlog untuk output PostgreSQL. Jika gangguan berlanjut dan batas tercapai, event lama dapat hilang sehingga kronologi log tidak lengkap.
6. **Bagaimana mendeteksi duplikasi record setelah retry?**
   Cari event dengan kombinasi request_id, nama event, service, percobaan (attempt), dan isi yang sama, lalu hitung kemunculannya di logs.fluentbit.
7. **Kapan PostgreSQL tidak lagi sesuai sebagai backend log utama?**
   Ketika volume masuk, masa retensi, atau kebutuhan pencarian membuat penyimpanan dan query log mengganggu kinerja serta kapasitas PostgreSQL. Pada kondisi itu perlu mengevaluasi backend log khusus; proyek ini memakai PostgreSQL untuk pembelajaran query dan investigasi, bukan sebagai ukuran kesiapan skala produksi.
8. **Kontrol apa yang perlu ditambahkan sebelum produksi?**
   Gunakan HTTPS/TLS dan secure cookie, pengelolaan serta rotasi secret, pembatasan akses ke log, kebijakan retensi dan backup, pemantauan kapasitas buffer serta kegagalan pengiriman, dan mekanisme penanganan duplikasi.